

ECE/CS/ME 539: Activity

Single Layer Neural Networks

Problem 1: True/False

- (a) The McCulloch-Pitts neuron model uses the threshold function.
- (b) The linear regression model is equivalent in its architecture to a single neuron model without an activation function.
- (c) The logistic regression model is equivalent in its architecture to a single neuron model with a hyperbolic tangent activation.
- (d) $\text{ReLU}(-5) = 0$
- (e) $\text{TanH}(-3) = -\text{TanH}(3)$
- (f) $\lim_{x \rightarrow -\infty} \text{Sigmoid}(x) = -1$
- (g) $\lim_{x \rightarrow \infty} \frac{d}{dx} \text{Sigmoid}(x) = 1$
- (h) Increasing the temperature T of the Sigmoid activation, makes the transition between the “off” ($f(x)=0$) and “on” ($f(x)=1$) states faster.

Problem 2: Derivatives of Activation Functions

Let $f(x) : \mathfrak{R} \rightarrow \mathfrak{R}$ be an activation function. Compute the derivatives $df(x)/dx$ for the activation functions below.

Activation Function	$f(x)$	$f'(x) = df(x)/dx$
ReLU	$\max(0, x)$	
Sigmoid	$\frac{1}{1 + e^{-x/T}}$	
Hyperbolic tangent	$\frac{e^{x/T} - e^{-x/T}}{e^{x/T} + e^{-x/T}}$	
Inverse tangent	$\frac{2}{\pi} \tan^{-1} \left(\frac{x}{T} \right)$	

Problem 3: Gradient Descent for Logistic Regression

In this problem, we will walk through a single step of the gradient descent algorithm for logistic regression. We will assume a n dimensional input $\bar{x}$. After including a bias parameter for the first dimension $x = [1, \bar{x}]$, we have $\theta \in \mathbb{R}^{n+1}$ and $x \in \mathbb{R}^{n+1}$. As a reminder,

$$\hat{y} = f(x, \theta)$$

$$f(x; \theta) = \sigma(\theta^\top x) = \frac{1}{1 + \exp(-\theta^\top x)}$$

$$\text{Cross entropy loss } L(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

$$\text{The single update step } \theta^{t+1} = \theta^t - \eta \nabla_\theta L(f(x; \theta), y)$$

- Find the expression for the gradient $\nabla_\theta L(f(x; \theta), y)$. The expression should only depend on the model parameters θ , the input x , the predicted output $\hat{y}$, and the true output y .
- Compute the updated parameters θ^1 , given the following initial parameters and data points.

$$\text{Initial parameters: } \theta^0 = [0, 0, 0]$$

$$\text{Learning rate } \eta = 0.1$$

$$\text{data example: } x = [1, 3, 2], y = 1$$

Problem 4: Gradient Descent (Application)

We will use PyTorch to perform data adaptive training (aka mini-batch stochastic gradient descent) to learn a simple linear regression model to minimize a least squares objective.

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^T \mathbf{w} + b$$

$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} \frac{1}{2N} \sum_{i=1}^N \|y_i - f(\mathbf{x}_i; \mathbf{w}, b)\|_2^2$$

Data adaptive training finds the optimal parameters by gradient descent. In other words, it starts from a random guess for the model parameters, $\mathbf{w}^{(0)}, b^{(0)}$, and at each iteration it takes a small step in the direction of the negative gradient.

$$\begin{aligned} \mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w}^{(k)}, b^{(k)}) \\ b^{(k+1)} &= b^{(k)} - \alpha \nabla_b L(\mathbf{w}^{(k)}, b^{(k)}) \end{aligned}$$

where $L(\mathbf{w}^{(k)}, b^{(k)})$ is the loss function for the model at iteration k and α is the learning rate.

- Review the provided starter code carefully (`Ex_SNM_DAT_torch_starter.ipynb`). This code is complete. It loads the iris dataset, splits it into training and testing data. It then defines several helper functions to compute the model predictions, compute the loss, update the parameters, and compute the evaluation metric R^2 . Finally, the code uses these functions to train the model.
- After creating a copy of the code, modify it to compute both the training and testing R^2 at the end of each epoch. Plot the training R^2 vs epoch and test R^2 vs epoch. Discuss your observations.

- (c) Try different learning rates (use `lr=0.03,0.01,0.003,0.001`). Plot the test R^2 vs epoch for each run. Discuss your observations.
- (d) Try different batch sizes (use `batch_size=1,3,10,30,100`). Plot R^2 vs epoch for each run. Discuss your observations.